VISIONQUEUE

Project Handoff & Execution Summary

Use this document with the latest PRD, TDD and Team Allocation document when starting the execution chat.

1. Why This Document Exists

This is a handoff summary of the decisions, architecture principles, scope boundaries and execution approach established before implementation. It is intentionally not a replacement for the PRD or TDD. The latest PRD remains the functional source of truth, the TDD remains the technical blueprint, and the team-allocation document remains the personnel/work-allocation reference.

Recommended handoff package for the next ChatGPT conversation: (1) latest approved PRD, (2) latest TDD, (3) Team Work Allocation & Role Guide, and (4) this summary. The new chat should read all four before giving implementation instructions.

2. Project Identity

Project: VisionQueue — Intelligent Queue and Crowd Management System.

Context: College Computer Vision Laboratory project.

Purpose: Build a real, working, product-like, single-camera computer-vision system for common public environments such as college facilities, counters, waiting areas, entrances, service areas and queues.

The current version is deliberately practical and locally executable. It is not a toy 'YOLO detects people' demo and it is not an enterprise cloud platform.

3. Current V1 Direction

- Single camera.
- Local processing on a Windows computer.
- Real-time operation with smoothness and freshness prioritized over processing every captured frame.
- Person detection, tracking, counting and queue/crowd monitoring.
- Current occupancy/count and entry/exit counting.
- Approximate session-unique counting where supported, with limitations stated honestly.
- Queue/crowd analytics and useful basic alerts/status.
- Live dashboard.
- Local history where included by the approved PRD.
- Graceful camera, GPU/runtime, backend and WebSocket failure states.
- Privacy-conscious design: no face recognition or persistent biometric identity in V1.
- Mostly/free-cost technology choices.

4. Current V1 Architecture

The agreed high-level flow is:

Camera → OpenCV → Person Detection → Tracking → Counting → Queue/Crowd Analytics → Application State → FastAPI → WebSocket/REST → React Dashboard, with SQLite used for appropriate persistent history/configuration.

Face detection is a separate CV pipeline and is not the primary counting mechanism.

Face detection is not face recognition. V1 should not introduce names, face embeddings, biometric databases or persistent biometric identity unless the approved PRD is changed and technically justified.

5. Real-Time Design Principles

- Use bounded/latest-frame processing rather than allowing an unlimited backlog of old frames.
- Separate the critical CV loop from database writes and frontend communication.
- Prefer fresh information over processing increasingly stale frames.
- Track capture timestamps and monitor frame age/processing delay.
- Use asynchronous/non-critical operations where appropriate.
- Do not make the pipeline Camera → inference → database write → API → UI → next frame.
- Treat FPS, latency and resource usage as targets to validate experimentally, not guaranteed numbers.
- Do not fabricate zero counts when the camera is offline or the system is degraded.

6. Technology Direction

The current working V1 baseline established during the TDD work is:

- Python 3.11.x
- OpenCV 5.x
- YOLO26n for person detection
- YuNet for face detection
- ByteTrack for tracking
- ONNX Runtime GPU execution on GPU-capable systems, with CPU fallback where feasible
- OpenCV VideoCapture / compatible camera input
- FastAPI + Uvicorn
- WebSocket for low-volume live metrics/state
- SQLite + WAL
- React + TypeScript
- Vite
- Tailwind CSS
- Recharts
- TanStack Query
- Git + GitHub
- pytest / API and integration testing
- Vitest + React Testing Library where appropriate

Important runtime correction: do not install both ONNX Runtime CPU and GPU Python packages in the same environment. Use the appropriate runtime for the environment, with the application architecture supporting GPU/CPU capability selection/fallback.

Do not add Redis, PostgreSQL, Kafka, RabbitMQ, Kubernetes, cloud GPUs, vector databases or unnecessary microservices to V1 unless an actual requirement emerges.

7. Why V1 Is Local-First

- The approved V1 is a single-camera local-processing system.
- Local inference avoids continuous camera-stream upload latency and Internet dependency.
- The project already has access to a suitable local GPU-capable development environment.
- Local processing reduces recurring cloud GPU costs.
- Local processing is simpler to demonstrate, debug and deploy for the college project.
- Local processing reduces unnecessary transmission of camera imagery.
- The architecture should remain modular/cloud-ready without making cloud infrastructure a V1 dependency.

Future cloud/edge architecture is appropriate only when requirements such as multiple cameras, multiple locations, centralized monitoring, distributed processing or SaaS-style deployment justify it.

8. Redis Decision

Redis is intentionally not part of V1. Current live state can be held in application memory and pushed through WebSocket, while SQLite provides appropriate local persistence. Redis becomes useful later if VisionQueue evolves into a distributed multi-camera/multi-location architecture requiring shared live state, pub/sub, distributed workers or similar capabilities.

This is a 'not required now' decision, not a statement that Redis is a bad technology.

9. Team Work Allocation

The detailed personnel allocation is in the separate Team Work Allocation & Role Guide. The six members are:

- Ram Samhith — Computer Vision & System Integration.
- Shaik Shakeeb — Computer Vision & Tracking.
- B. Vijay Kumar Goud — Backend & API.
- Y. Manoj Kumar Reddy — Database & Data Management.
- V. Phaneeth Kumar — Frontend & Dashboard.
- P. Roony Williams — Testing & Integration.

The TDD intentionally does not contain personal names, individual hardware specifications, leadership labels or personnel assignments.

10. Ram's Working Approach

Ram can work mostly independently. The architecture was deliberately designed to minimize unnecessary coupling.

- Build the CV side independently: camera input, detection, face detection, tracking/counting integration, ROI/line crossing, CV state, runtime/fallback and performance validation.
- Coordinate with Shakeeb mainly around the detection → tracking interface and tracking behavior.
- Coordinate with Vijay when the CV/analytics output is connected to the backend application-state/API contract.
- Provide Roony with repeatable CV test inputs, expected outputs and known limitations for testing.
- Do not continuously edit frontend, backend or database code merely to keep everyone connected.

Target collaboration pattern: roughly 80–90% independent work with short, purposeful interface/integration coordination.

11. Ram ↔ Shakeeb Connection

Do not build the entire pipeline together from day one. Work independently and connect through a stable interface.

Conceptual flow:

Ram's detection module → detection contract → Shakeeb's ByteTrack module → track contract → counting/analytics.

A detection object should carry the information needed by the tracker, such as bounding box, confidence and class. A tracking output should carry a track ID, bounding box and confidence plus any required tracking metadata. Exact Python class/function signatures should be finalized during implementation after the actual model APIs are tested; the TDD is the architecture/contract reference, not a reason to prematurely freeze every internal function.

12. Module Independence / Mock Mode

- Frontend can work with mock REST/WebSocket data.
- Backend can work with simulated CV/analytics data.
- Database can be tested using simulated sessions/measurements/events.
- Analytics can be tested without a live camera.
- CV can be tested independently with video files.
- Testing can validate interfaces independently.
- Mock data should follow the same application-level data contracts as the real pipeline.

This prevents weaker development machines or unfinished modules from blocking the whole team.

13. Integration Principle

No module should depend on another member's internal implementation. Modules communicate through stable contracts.

The intended chain is:

CV → Analytics → Application State → Backend/API → Frontend, with Database persistence connected where appropriate.

- CV must not directly control the frontend.
- Frontend must not directly access CV implementation.
- Database must not be in the critical per-frame inference path.
- Backend request handlers must not contain heavy CV inference.
- WebSocket is for live metrics/state, not full video transport.

14. Current Work Status

Completed before execution:

- Project concept and practical V1 scope established.
- College-friendly abstract/front-page work completed.
- PRD direction established and latest PRD treated as primary functional source.
- TDD created and refined with real-time architecture, interfaces, failure handling, testing and technology analysis.
- Technology stack researched and narrowed to a practical local-first V1.
- Team allocation created separately.
- Hardware heterogeneity accounted for generically in the architecture.
- Decision made to avoid unnecessary cloud/distributed infrastructure in V1.
- Decision made to use mock/simulation interfaces for independent development.
- Decision made that Ram can work mostly independently with limited integration points.

15. What Is Still NOT 'Finished'

- Exact implementation-level Python interfaces/classes/functions are not yet frozen.
- Actual model/runtime installation and compatibility must be validated on the development machine.
- Actual FPS, latency, memory and resource usage are not yet measured.
- Actual detection/tracking/counting behavior still needs testing on representative videos/camera scenes.
- ROI geometry and practical camera placement must be validated in the target environment.
- Counting and session-unique behavior need real-world validation.

- Exact thresholds/parameters marked as open in the PRD/TDD should be experimentally selected rather than guessed.
- Git repository/branching workflow and local development environments still need to be set up.
- End-to-end integration has not started yet.

16. Recommended Execution Order in the New Chat

1. Give the new chat the latest PRD, latest TDD, Team Work Allocation & Role Guide and this summary.
2. Ask the new chat to read all four documents and first produce a concise execution baseline, not code.
3. Confirm the exact V1 scope and identify only the genuinely unresolved validation items.
4. Set up Git/GitHub repository and branch workflow.
5. Set up the development environment and verify Python, camera access and basic OpenCV operation.
6. Verify the inference runtime and model loading before building the complete CV pipeline.
7. Build camera/input abstraction and a repeatable video-file test mode.
8. Build person detection and validate raw detections.
9. Integrate tracking and validate track stability.
10. Build ROI/line-crossing and counting logic.
11. Build analytics/application-state output.
12. Build mock mode/contracts so backend/frontend work independently.
13. Implement FastAPI/REST/WebSocket and SQLite layers.
14. Implement React dashboard.
15. Integrate all modules.
16. Run performance, failure, regression and long-duration tests.
17. Document measured results and known limitations.

17. What Should NOT Happen Before Execution

- Do not keep rewriting the PRD/TDD without a concrete reason.
- Do not add Redis/cloud/Kafka/microservices merely because they look advanced.
- Do not prematurely optimize before measuring.
- Do not freeze exact internal class/function structures before checking actual library APIs.
- Do not build all six areas sequentially if they can use mocks and contracts in parallel.
- Do not claim accuracy/FPS/latency before measuring it.
- Do not make the entire team dependent on the strongest GPU machine.
- Do not attempt advanced multi-camera/cloud/AI-assistant features during V1 implementation.

18. First Execution Checklist

- ☐ Latest PRD attached and identified as functional source of truth.
- ☐ Latest TDD attached and identified as technical blueprint.
- ☐ Team allocation document attached.
- ☐ This handoff summary attached.
- ☐ GitHub repository/branch strategy agreed.
- ☐ Python environment created.
- ☐ OpenCV camera access verified.
- ☐ Video-file test input prepared.
- ☐ ONNX Runtime execution provider/runtime verified.

- ☐ YOLO26n model loading verified.
- ☐ YuNet model availability/loading verified.
- ☐ ByteTrack integration approach verified.
- ☐ Basic logging/configuration established.
- ☐ Initial CV module interface agreed.

19. Handoff Instruction for the Next Chat

The next chat should NOT restart project planning from zero. It should treat the attached latest PRD as the functional source of truth, the latest TDD as the technical blueprint, the Team Work Allocation & Role Guide as the personnel plan, and this document as the context/decision summary.

The next task should be execution preparation and then implementation in small, testable stages. Before coding, verify the actual environment and current library/model compatibility. Do not invent missing requirements; ask only when a genuine ambiguity blocks implementation. Keep V1 local-first, real-time, modular, testable and feasible.

20. Bottom Line

The planning phase is sufficiently mature to start execution. What remains should mostly be validated through implementation and testing rather than another large planning cycle. The next chat should therefore move from 'what should we build?' to 'let's verify the environment, create the repository structure, establish the first interfaces, and build the system incrementally.'